

Universidad Nacional
Autónoma de México

Facultad de Ingeniería



---

# Sistemas Operativos

Investigación del tema:

## Sistemas Operativos en Sistemas Embebidos


**Alumnos:**

Casas Pérez Rodrigo Imanol [321183895]

Malfavon Lemus Paris Samantha [321249081]

**Profesor:**

Gunnar Eyal Wolf Iszaevich

**Grupo:**

7


---


Facultad de Ingeniería, UNAM
Semestre 2027-1

# Índice

1. Introducción

2. Conceptos fundamentales

2.1 Sistema embebido, firmware y sistema operativo

2.2 ¿Cuándo se necesita un sistema operativo?

3. Características y restricciones

4. Arquitectura de un sistema operativo embebido

4.1 Componentes principales

4.2 Secuencia de arranque

5. Clasificación y ejemplos de sistemas operativos embebidos

5.1 Sistemas de tiempo real

5.2 Linux embebido y otras alternativas

6. Administración de tareas, planificación y memoria

6.1 Tareas, prioridades e interrupciones

6.2 Memoria y sincronización

7. Linux en sistemas embebidos

7.1 Componentes y herramientas de construcción

7.2 Ejemplo de un router

8. Aplicaciones en la vida cotidiana y la industria

9. Ventajas, limitaciones y seguridad

10. Conclusiones

11. Referencias

# 1. Introducción

Los sistemas embebidos son sistemas de cómputo integrados en productos cuyo objetivo principal no es ofrecer una computadora de propósito general, sino controlar, medir, comunicar o procesar información para cumplir funciones determinadas. Se encuentran en equipos de telecomunicaciones, vehículos, aparatos domésticos, instrumentos médicos y sistemas industriales. Su presencia es importante porque numerosas actividades cotidianas dependen de software que opera de manera continua y, en ocasiones, bajo restricciones estrictas de tiempo y recursos.

Dentro de este ámbito, el sistema operativo embebido administra el procesador, la memoria y los dispositivos de entrada y salida, además de proporcionar mecanismos para ejecutar varias tareas y comunicarlas. Sin embargo, no todos los sistemas embebidos necesitan uno: algunos microcontroladores ejecutan directamente una aplicación sobre el hardware. La elección depende de la complejidad del producto, sus requisitos de respuesta y los recursos disponibles.

La presente investigación describe los conceptos, la arquitectura y el funcionamiento de estos sistemas, con atención particular a Linux embebido y a los sistemas operativos de tiempo real. El propósito es relacionar sus aplicaciones prácticas con temas de la asignatura de Sistemas Operativos, como planificación, memoria, interrupciones y controladores de dispositivos.

# 2. Conceptos fundamentales

## 2.1 Sistema embebido, firmware y sistema operativo

Sistema embebido: combinación de hardware y software incorporada en un producto para realizar funciones definidas. Puede emplear un microcontrolador o un procesador de aplicaciones, junto con sensores, actuadores y dispositivos de comunicación.

Firmware: software estrechamente asociado con el funcionamiento del dispositivo, generalmente almacenado en memoria no volátil. El término puede referirse al programa completo de un microcontrolador o a componentes de bajo nivel, como el software de arranque.

Sistema operativo embebido: conjunto de componentes de software que proporciona servicios para administrar recursos, organizar la ejecución y ofrecer interfaces a las aplicaciones. Puede ser un núcleo de tiempo real pequeño o un sistema más amplio basado en Linux.

## 2.2 ¿Cuándo se necesita un sistema operativo?

Un dispositivo que solamente lee un sensor y activa una salida puede ejecutar un ciclo principal sin sistema operativo, conocido como programación bare-metal. En cambio, si debe atender comunicaciones de red, registrar datos, actualizar una interfaz y controlar periféricos de manera concurrente, un sistema operativo puede simplificar la organización del software. Esto no implica que todos los dispositivos complejos utilicen

Linux: en equipos con memoria muy limitada o requisitos temporales estrictos puede resultar más adecuado un RTOS.

## 3. Características y restricciones

La característica principal de un sistema embebido es su diseño para un conjunto concreto de funciones. A partir de ello aparecen restricciones que varían según el producto:

• Recursos: capacidad de procesamiento, RAM y almacenamiento dimensionados según la aplicación.

• Energía: necesidad de reducir el consumo, especialmente en equipos alimentados por baterías.

• Confiabilidad: funcionamiento estable durante largos periodos y recuperación ante fallos.

• Tiempo de respuesta: algunas acciones deben completarse antes de un plazo definido; otras admiten mayor variación.

• Integración: interacción directa con sensores, buses, controladores y actuadores.

Estas restricciones explican por qué no siempre conviene instalar una distribución de escritorio en un dispositivo dedicado. En muchos casos se elimina software innecesario y se construye una imagen del sistema específica para el hardware.

## 4. Arquitectura de un sistema operativo embebido

### 4.1 Componentes principales

Una arquitectura típica basada en Linux puede entenderse mediante cuatro niveles. En la base se encuentra el hardware (CPU, memoria, almacenamiento y periféricos). Sobre él trabaja el kernel, responsable de la planificación, memoria, controladores, interrupciones y llamadas al sistema. Encima aparecen las bibliotecas y servicios de usuario, que ofrecen funciones comunes. Finalmente, las aplicaciones realizan la tarea específica del producto. En un RTOS pequeño, algunas de estas capas pueden estar combinadas o no existir como procesos separados.

| Nivel | Función |
|---|---|
| Aplicación | Control de sensores, comunicación, interfaz y lógica del producto. |
| Servicios y bibliotecas | Herramientas, bibliotecas y demonios requeridos por la aplicación. |
| Kernel | Planificación, memoria, controladores y mecanismos de comunicación. |
| Hardware | Procesador, memoria, interfaces y dispositivos físicos. |

### 4.2 Secuencia de arranque

En muchos dispositivos Linux, al encender el equipo se ejecuta primero código de arranque y posteriormente un bootloader, como U-Boot. Este prepara las condiciones necesarias para cargar el kernel y, según la plataforma, proporciona una descripción del hardware mediante Device Tree. El kernel inicializa sus subsistemas, monta el sistema de archivos raíz y ejecuta el primer proceso de usuario, normalmente init (PID 1), que inicia los servicios y la aplicación principal. Esta secuencia es representativa y puede cambiar según el hardware y el diseño del producto.

Encendido → firmware/bootloader → kernel Linux → sistema de archivos raíz → init → aplicación

## 5. Clasificación y ejemplos de sistemas operativos embebidos

### 5.1 Sistemas operativos de tiempo real (RTOS)

Un sistema operativo de tiempo real está orientado a que determinadas operaciones se completen dentro de límites temporales establecidos. Su cualidad principal no es ejecutar siempre más rápido, sino ofrecer comportamiento temporal predecible. En un sistema de tiempo real estricto, incumplir un plazo puede constituir un fallo del sistema; en uno de tiempo real flexible, el retraso puede reducir la calidad del servicio sin implicar necesariamente un fallo catastrófico.

FreeRTOS es un ejemplo de RTOS utilizado en microcontroladores. Su planificador puede seleccionar la tarea lista de mayor prioridad y desplazar a otra cuando se requiere. Zephyr es otra plataforma de tiempo real dirigida a dispositivos conectados y de recursos restringidos. [1, 2]

### 5.2 Linux embebido y otras alternativas

Linux embebido designa sistemas construidos alrededor del kernel Linux y adaptados a un dispositivo concreto. Resulta útil cuando se necesitan protocolos de red, sistemas de archivos, controladores y aplicaciones relativamente complejas. No se trata de un sistema operativo completamente distinto del Linux usado en servidores o computadoras: comparte el kernel, pero cambia la selección y configuración de componentes. Linux puede incorporar capacidades de tiempo real, aunque su idoneidad depende de los plazos y de la configuración concreta.

| Alternativa | Uso típico | Característica destacada |
|---|---|---|
| FreeRTOS | Microcontroladores | Planificación de tareas y tamaño reducido. |
| Zephyr | IoT y equipos restringidos | Servicios configurables y orientación a tiempo real. |
| Linux embebido | Routers, multimedia, industria | Ecosistema amplio de controladores, redes y procesos. |

# 6. Administración de tareas, planificación y memoria

## 6.1 Tareas, prioridades e interrupciones

Un sistema embebido puede ejecutar varias actividades lógicas: leer sensores, atender mensajes, controlar una salida y almacenar información. El planificador decide cuál utiliza el procesador en cada instante. En muchos RTOS se emplea planificación expulsiva por prioridades: cuando una tarea de prioridad superior está lista, puede interrumpir la ejecución de una tarea de prioridad inferior. El funcionamiento exacto depende de la política configurada y del número de núcleos. [1]

Las interrupciones permiten reaccionar ante eventos de hardware sin comprobar constantemente el estado de cada periférico. La rutina de atención debe mantenerse breve y, cuando corresponda, despertar una tarea para realizar el trabajo más largo. Por ejemplo, un sensor puede generar una interrupción que active una tarea encargada de procesar una medición.

## 6.2 Memoria y sincronización

La gestión de memoria depende de la plataforma. En un microcontrolador pequeño puede no existir unidad de administración de memoria (MMU), por lo que las tareas comparten un espacio de direcciones y se busca controlar cuidadosamente el uso de pilas y memoria dinámica. En sistemas Linux con MMU, los procesos normalmente disponen de espacios de direcciones virtuales separados, lo que facilita el aislamiento.

Cuando varias tareas utilizan recursos compartidos, se necesitan mecanismos de sincronización. Los mutex protegen secciones críticas, los semáforos coordinan eventos o disponibilidad de recursos y las colas permiten intercambiar mensajes. Un mal diseño puede provocar condiciones de carrera, interbloqueos o inversión de prioridades; esta última ocurre cuando una tarea de alta prioridad queda bloqueada indirectamente por otra de menor prioridad.

# 7. Linux en sistemas embebidos

## 7.1 Componentes y herramientas de construcción

Un sistema Linux embebido suele integrar un bootloader, una imagen del kernel, un sistema de archivos raíz, bibliotecas, servicios y aplicaciones. La imagen final se adapta a la arquitectura del procesador y al hardware disponible. Es frecuente emplear compilación cruzada: el software se compila en una computadora de desarrollo y se ejecuta en otra arquitectura, por ejemplo ARM o RISC-V.

Buildroot automatiza la generación de la cadena de compilación, el sistema de archivos raíz, el kernel y, cuando se requiere, el bootloader. Su enfoque facilita producir imágenes relativamente compactas para dispositivos específicos. [3]

Yocto Project proporciona herramientas y prácticas para crear distribuciones Linux personalizadas. Su modelo de capas permite separar configuraciones del hardware,

paquetes, políticas de distribución y aplicaciones, lo que favorece la reutilización y el mantenimiento de productos complejos. [4]

El kernel Linux proporciona las interfaces y subsistemas básicos, pero por sí solo no constituye toda la distribución. Para que un producto funcione también se requieren componentes de espacio de usuario y una configuración adecuada del arranque y del almacenamiento. [5]

### 7.2 Ejemplo: router doméstico

Un router puede incorporar un procesador, interfaces Ethernet y Wi-Fi, memoria y almacenamiento flash. Un sistema basado en Linux puede encargarse de la pila de red, controladores, reglas de filtrado, configuración y servicios de administración. A diferencia de una computadora de escritorio, normalmente no requiere un entorno gráfico completo ni aplicaciones de oficina. Su sistema se prepara para iniciar automáticamente los servicios de conectividad y mantenerlos operativos.

## 8. Aplicaciones en la vida cotidiana y la industria

Los sistemas embebidos aparecen en sectores muy diversos. Los routers y puntos de acceso gestionan paquetes y conexiones; los televisores inteligentes ejecutan interfaces y servicios multimedia; los vehículos integran unidades de control y sistemas de infoentretenimiento; los equipos industriales supervisan variables y accionan dispositivos; y los instrumentos médicos pueden medir señales o coordinar subsistemas. El sistema operativo seleccionado depende de la función: una interfaz multimedia puede beneficiarse de Linux, mientras que un controlador con plazos críticos puede requerir un RTOS o una arquitectura especializada.

Es importante no confundir el hecho de que un producto contenga componentes electrónicos programables con que necesariamente ejecute Linux o cualquier otro sistema operativo. Incluso dentro de un mismo automóvil pueden coexistir controladores sin SO, microcontroladores con RTOS y procesadores con Linux.

## 9. Ventajas, limitaciones y seguridad

Entre las ventajas de emplear un sistema operativo embebido están la reutilización de controladores, la administración estructurada de tareas, la disponibilidad de mecanismos de comunicación y la posibilidad de mantener aplicaciones complejas. Linux aporta además un ecosistema amplio de herramientas y software libre. En contraste, un sistema operativo también consume memoria y procesamiento, aumenta la complejidad de configuración y puede introducir dependencias que no son necesarias en aplicaciones sencillas.

La seguridad requiere atención especial porque muchos dispositivos permanecen conectados a redes durante años. Es necesario limitar servicios expuestos, proteger credenciales, controlar privilegios, aplicar actualizaciones verificadas y prever mecanismos de recuperación. También se debe considerar el mantenimiento del kernel

y de las bibliotecas: un producto sin actualizaciones puede conservar vulnerabilidades conocidas aun cuando continúe funcionando correctamente.

En el diseño de un producto deben evaluarse conjuntamente requisitos de tiempo real, costo, memoria, consumo, disponibilidad de controladores, mantenimiento y seguridad. No existe un sistema operativo óptimo para todos los escenarios.

## 10. Conclusiones

Los sistemas operativos embebidos permiten organizar y controlar el software de dispositivos especializados. Su importancia se entiende al relacionar las restricciones físicas del equipo con las funciones del sistema operativo: planificación, administración de memoria, interrupciones, controladores y sincronización. En dispositivos sencillos puede ser suficiente un programa sin sistema operativo; en otros, un RTOS aporta mecanismos de respuesta temporal; y en equipos con servicios de red y aplicaciones complejas, Linux embebido ofrece una plataforma flexible.

El uso de Linux no elimina la necesidad de diseñar para el hardware disponible. Es necesario seleccionar componentes, preparar el arranque, construir el sistema de archivos y mantener actualizaciones. Por ello, la elección del sistema operativo debe partir de los requisitos concretos del dispositivo y no solamente de la popularidad de una tecnología.

## 11. Referencias


Amazon Web Services. (s. f.). *RTOS fundamentals*. FreeRTOS. https://www.freertos.org/Documentation/01-FreeRTOS-quick-start/01-Beginners-guide/01-RTOS-fundamentals

Zephyr Project. (s. f.). *Zephyr Project documentation*. https://docs.zephyrproject.org/latest/

Buildroot Developers. (s. f.). *The Buildroot user manual*. https://buildroot.org/downloads/manual/manual.html

Yocto Project. (s. f.). *Overview and concepts manual*. https://docs.yoctoproject.org/overview-manual/

The Linux Kernel Development Community. (s. f.). *The Linux kernel documentation*. https://docs.kernel.org/